



# MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY ALLAHABAD

Prayagraj, Uttar Pradesh, India

**Department of Electronics and Communication Engineering**


---

# P R O J E C T  R E P O R T

*on*

# SPI EEPROM MOTOR-PROFILE CONTROLLER

*Storage, Retrieval and Execution of DC Motor Speed Profiles using an LPC2138 Microcontroller, a 25LC256 SPI EEPROM and an L293D Motor Driver*

| | |
|---|---|
| **Project** | SPI EEPROM Motor-Profile Controller |
| **Microcontroller** | NXP LPC2138 / LPC2148 (ARM7TDMI-S) |
| **Profile memory** | Microchip 25LC256 SPI EEPROM, six motor profiles |
| **Motor stage** | L293D H-bridge driver and 12 V DC motor |
| **Display** | 16×2 LCD (LM016L) and status LED |
| **Firmware / tools** | Embedded C, Keil µVision (ARM compiler) |
| **Simulation** | Proteus 8 Professional |


## SUBMITTED BY

Team 1

| S. No. | Name of Student | Roll Number |
|---|---|---|
| 1 | Anurag Nirmal | 20244031 |
| 2 | Arin Saraf | 20244034 |
| 3 | Arun Kumar | 20244038 |
| 4 | Ashish Kumar Kashyap | 20244043 |
| 5 | Ashish Pandey | 20244044 |

## SUBMITTED TO

**Dr. Shishir Maheshwari**

Department of Electronics and Communication Engineering

Motilal Nehru National Institute of Technology Allahabad


---

**Academic Session: 2026–2027**

# ABSTRACT

This report presents an **SPI EEPROM Motor-Profile Controller** built around the NXP **LPC2138** ARM7 microcontroller (the firmware also runs on the LPC2148). Six motor profiles that imitate everyday appliances — demonstration, washing-machine wash and spin-dry, gate opening and closing, natural-breeze fan and blender pulse — are stored in a Microchip **25LC256** SPI EEPROM, one per 64-byte page, each protected by a version, an id and a checksum.

At power-up the firmware reads and checks every page, restores any blank or corrupt page from built-in defaults, and runs the profiles in sequence on a DC motor through an **L293D** H-bridge with 100 Hz PWM, linear speed ramps and a 200 ms dead time before every reversal. A **16×2 LCD** shows the profile name, direction, speed, phase, step and repeat, and a status LED uses different blink patterns for forward, reverse, stop and ramping. All motor events are scheduled on one free-running timer while the LCD, LED and an optional NEXT button are serviced in the background, so timing is exact and does not drift. The design was built in Keil µVision and verified in Proteus, where the EEPROM contents matched the firmware's records byte for byte, and in an instruction-level emulator, which confirmed the full 88.2 s sequence. Testing on physical hardware remains future work.

**Keywords:** LPC2138, LPC2148, ARM7TDMI-S, SPI, 25LC256 EEPROM, L293D, PWM, 16×2 LCD, appliance motor profiles, Keil µVision, Proteus simulation.

# ACKNOWLEDGEMENT

We express our sincere gratitude to **Dr. Shishir Maheshwari**, Department of Electronics and Communication Engineering, Motilal Nehru National Institute of Technology Allahabad, for his guidance and for giving us the opportunity to work on this project.

We also thank the Department of Electronics and Communication Engineering, MNNIT Allahabad, for providing the academic environment and resources that made this work possible. Finally, we thank our classmates and families for their encouragement throughout the project.

**Team 1**
Department of ECE, MNNIT Allahabad

# TABLE OF CONTENTS

**ABSTRACT** ..... **i**
**ACKNOWLEDGEMENT** ..... **i**
**1. INTRODUCTION** ..... **1**
1.1 Project Overview ..... 1
1.2 Motivation and Scope ..... 1
1.3 Development Environment ..... 1
**2. OBJECTIVES** ..... **1**
**3. TECHNICAL BACKGROUND** ..... **2**
3.1 LPC2138 / LPC2148 Microcontroller ..... 2
3.2 SPI Interface ..... 2
3.3 25LC256 Serial EEPROM ..... 2
3.4 PWM Motor Control with the L293D ..... 3
3.5 16×2 Character LCD ..... 3
**4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN** ..... **3**
4.1 Block Diagram ..... 3
4.2 Components and Pin Assignment ..... 4
4.3 Power Supply ..... 4
4.4 Proteus Schematic ..... 5
**5. MOTOR PROFILES AND EEPROM FORMAT** ..... **5**
5.1 Daily-Life Motor Profiles ..... 5
5.2 EEPROM Page Format ..... 6
**6. WORKING PRINCIPLE AND DESIGN CALCULATIONS** ..... **7**
6.1 Operating Sequence ..... 7
6.2 Design Calculations ..... 7
6.3 LED Status Patterns ..... 7
6.4 LCD Screen Layout ..... 8
**7. SOFTWARE DESIGN** ..... **8**
7.1 Build Process in Keil µVision ..... 8
7.2 Running the Simulation ..... 8
7.3 Firmware Structure ..... 9
7.4 Error Handling ..... 10
**8. SOURCE CODE** ..... **10**
8.1 Motor-Profile Controller (main.c) ..... 10
**9. RESULTS AND DISCUSSION** ..... **17**
9.1 Build and EEPROM Programming ..... 17
9.2 Profile Execution in Proteus ..... 17
9.3 LED and LCD Status over the Full Sequence ..... 19
9.4 Timing Verification ..... 21
9.5 Problems Faced and Solutions ..... 22
9.6 Discussion ..... 22
**10. APPLICATIONS AND FUTURE IMPROVEMENTS** ..... **22**
10.1 Applications ..... 22
10.2 Limitations and Future Improvements ..... 22
**11. CONCLUSION** ..... **23**
**REFERENCES** ..... **23**

## LIST OF FIGURES

Figure 4.1: System block diagram ..... 3
Figure 4.2: Proteus schematic of the complete system ..... 5
Figure 5.1: EEPROM page map and the layout of one profile page ..... 6
Figure 6.1: LED patterns over two seconds ..... 8
Figure 7.1: Firmware flowchart ..... 9
Figure 9.1: Keil µVision build output ..... 17
Figure 9.2: 25LC256 contents after the first run (Debug → SPI Memory) ..... 17
Figure 9.3: DEMO at 3.4 s (+342 RPM) and WASH at 29.7 s (+278 RPM) ..... 18
Figure 9.4: SPIN at 40.25 s (+404 RPM, rising) and GATE at 56.1 s (+371 RPM) ..... 18
Figure 9.5: MIXER at 104.0 s, slowing down after a pulse (+268 RPM) ..... 18
Figure 9.6: Measured motor drive over the full 88.2 s sequence (emulation) ..... 21

## LIST OF TABLES

Table 1.1: Development environment and tools ..... 1
Table 3.1: SPI0 registers and values used ..... 2
Table 3.2: 25LC256 instructions used ..... 2
Table 3.3: L293D control as used in this design ..... 3
Table 4.1: Components ..... 4
Table 4.2: Pin assignment ..... 4

Table 4.3: Supply connections....4
Table 4.4: Proteus MOTOR-DC properties used....4
Table 5.1: Motor profiles and their applications....5
Table 5.2: Steps of each profile (direction, speed %, ramp ms, run ms)....5
Table 5.3: Profile page byte layout....6
Table 6.1: Design calculations....7
Table 6.2: LED patterns....8
Table 7.1: Main firmware functions....9
Table 7.2: Fault codes....10
Table 9.1: Motor speed observed in Proteus....19
Table 9.2: LED and LCD status at different times (full sequence)....19
Table 9.3: Timing measured in emulation....21
Table 9.4: Main problems and how they were solved....22

# 1. INTRODUCTION

## 1.1 Project Overview

Many household appliances run a motor through a fixed pattern: a washing machine turns its drum forward and backward and then spins it fast, a gate opens, waits and closes, and a fan changes speed to imitate a natural breeze. This project builds one controller that can run all of these patterns. Each pattern, called a **motor profile**, is a short list of steps (direction, speed, ramp time and run time) stored in an external **25LC256 SPI EEPROM**.

At power-up the **LPC2138** microcontroller reads six profiles from the EEPROM, checks each one, and runs them on a DC motor through an **L293D** H-bridge. A **16×2 LCD** shows the profile name, direction, speed, phase, step and repeat count, and a **status LED** blinks in a different pattern for forward, reverse, stop and ramping. The firmware also runs unchanged on the LPC2148, which has the same peripherals at the same addresses.

The work was done in Keil µVision and Proteus 8 Professional on a Windows PC. Physical hardware was not built.

## 1.2 Motivation and Scope

- **One controller, many appliances:** the behaviour is defined by data in the EEPROM, not by separate programs.
- **Non-volatile settings:** profiles survive power loss and can be changed without reprogramming the microcontroller.
- **Clear status:** the LCD and LED show what the controller is commanding at every moment.
- **Safe motor control:** speed ramps, a 200 ms pause before every reversal, and IN1 and IN2 never driven together.
- **Exact timing:** every event is scheduled on one free-running timer, so a profile repeats with no drift.

Closed-loop speed control, editing profiles from a keypad or PC, and physical hardware are outside the present scope.

## 1.3 Development Environment

**Table 1.1:** *Development environment and tools*

| Item | Details |
|---|---|
| Host system | Windows PC |
| IDE and compiler | Keil µVision with the ARM compiler (ARM7 / LPC2100 device support) |
| Simulator | Proteus 8 Professional (LPC2138, 25LC256, L293D, LM016L and MOTOR-DC models) |
| Timing verification | Instruction-level emulator (Unicorn) with models of SPI0, the 25LC256, GPIO, Timer1 and the LCD controller |

# 2. OBJECTIVES

1. Configure the LPC2138 SPI0 interface at register level and write a complete driver for the 25LC256 EEPROM.
2. Define a checked EEPROM format that stores six motor profiles, one per 64-byte page, and restore defaults automatically when a page is blank or corrupt.

3. Design six practical profiles: washing-machine wash and spin, gate opening and closing, natural-breeze fan, blender pulse, and a demonstration profile.
4. Drive a DC motor through an L293D with 100 Hz PWM, linear ramps and a dead time before every reversal.
5. Show the controller state on a 16×2 LCD and with distinct LED blink patterns for forward, reverse, stop and ramping.
6. Keep all motor timing exact while the LCD, LED and button are being serviced.
7. Verify the design in Proteus and confirm the timing in an instruction-level emulator.

# 3. TECHNICAL BACKGROUND

## 3.1 LPC2138 / LPC2148 Microcontroller

The LPC2138 is a 32-bit ARM7TDMI-S microcontroller with 512 KB of flash and 32 KB of RAM. Its peripherals sit on the VPB bus, whose clock PCLK is derived from the CPU clock by the VPBDIV register. This design runs from a 12 MHz crystal with the PLL turned off, so CCLK = 12 MHz and, with VPBDIV = 0, PCLK = 3 MHz. The firmware uses SPI0, Timer1 and the GPIO pins of ports 0 and 1. The LPC2148 adds USB but keeps these peripherals and addresses, so the same program runs on both.

## 3.2 SPI Interface

SPI is a synchronous, full-duplex bus. The master drives the clock SCK and data line MOSI, receives on MISO, and selects the slave with an active-low chip select. In **mode 0** the clock idles low and data are sampled on the rising edge. Every byte sent also clocks one byte in. On the LPC2138, writing `S0SPDR` starts a transfer and the SPIF flag in `S0SPSR` marks its end. The chip select is an ordinary GPIO pin (P0.7) so that it can stay low for a whole multi-byte command.

**Table 3.1:** *SPI0 registers and values used*

| Register | Address | Value used |
|---|---|---|
| S0SPCR (control) | 0xE002 0000 | 0x20: master, CPOL = 0, CPHA = 0, MSB first |
| S0SPSR (status) | 0xE002 0004 | Read to test SPIF (bit 7), MODF and ABRT |
| S0SPDR (data) | 0xE002 0008 | Byte to send / byte received |
| S0SPCCR (clock) | 0xE002 000C | 8: SCK = 3 MHz / 8 = 375 kHz |

## 3.3 25LC256 Serial EEPROM

The 25LC256 holds 32 KB organised in 64-byte pages. A write must first be enabled with WREN, may not cross a page boundary, and takes up to 5 ms, during which the WIP bit of the status register stays set. Each profile in this project fits in one page, so one profile is written with a single page write.

**Table 3.2:** *25LC256 instructions used*

| Instruction | Opcode | Purpose |
|---|---|---|
| WREN | 0x06 | Set the write-enable latch (WEL) |
| RDSR | 0x05 | Read status: WIP (bit 0), WEL (bit 1), block protection (bits 3:2) |
| READ | 0x03 | Read data from a 16-bit address |
| WRITE | 0x02 | Write up to 64 bytes within one page |

### 3.4 PWM Motor Control with the L293D

The L293D contains two H-bridges. EN1 enables the first bridge and IN1/IN2 select the direction. Switching EN1 with a pulse-width-modulated signal sets the average motor voltage, and so the speed. Here EN1 is driven by software PWM at 100 Hz with 1 % resolution. Logic power (VSS) is 5 V and the motor supply (VS) is 12 V.

**Table 3.3:** *L293D control as used in this design*

| EN1 | IN1 | IN2 | Motor |
|---|---|---|---|
| PWM | 1 | 0 | Forward, speed set by duty cycle |
| PWM | 0 | 1 | Reverse, speed set by duty cycle |
| X | 0 | 0 | Outputs off, motor coasts |
| X | 1 | 1 | Never used (brake); prevented in firmware |

### 3.5 16×2 Character LCD

The LM016L is a 16-character, 2-line display with an HD44780-compatible controller. In **4-bit mode** each byte is sent as two nibbles on D4–D7, latched by a pulse on E, with RS selecting command or data. RW is tied to ground, so the busy flag cannot be read; instead the firmware waits at least 50 µs between bytes, which is longer than the 37 µs the controller needs.

## 4. SYSTEM ARCHITECTURE AND HARDWARE DESIGN

### 4.1 Block Diagram

Figure 4.1 shows the system. The microcontroller reads the profiles over SPI, drives the L293D with PWM and direction signals, writes status to the LCD on port 1 and blinks the LED on P0.1. An optional push-button on P0.15 skips to the next profile.

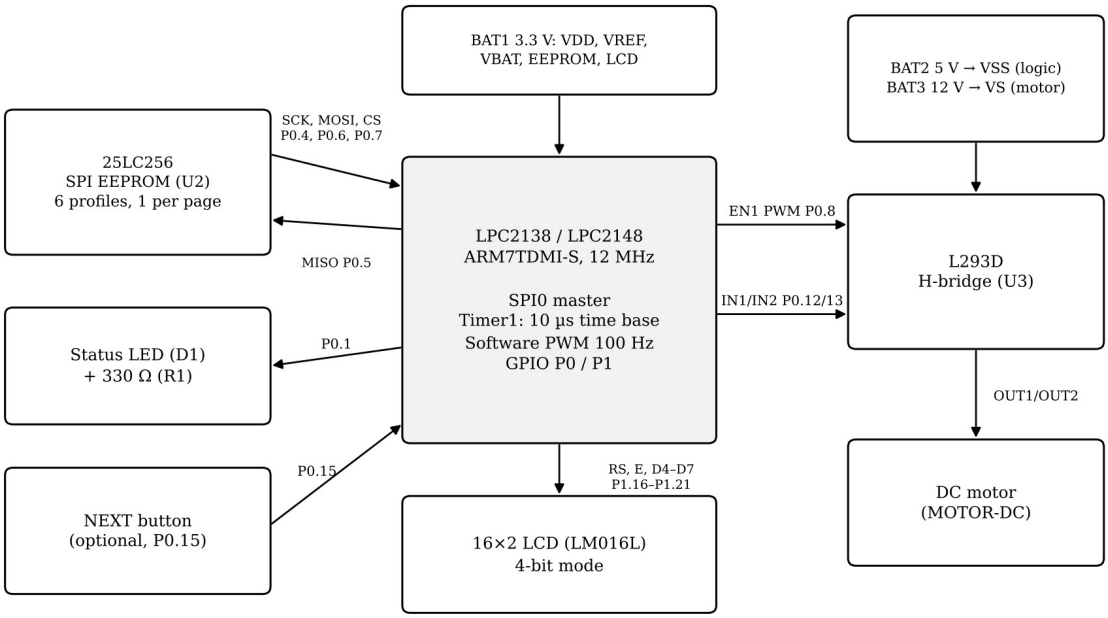



**Figure 4.1:** *System block diagram*

## 4.2 Components and Pin Assignment

**Table 4.1:** *Components*

| Ref. | Component | Function | Supply |
|---|---|---|---|
| U1 | LPC2138 (LPC2148 compatible) | Controller | 3.3 V |
| U2 | 25LC256 SPI EEPROM | Stores six motor profiles | 3.3 V |
| U3 | L293D | H-bridge motor driver | 5 V logic, 12 V motor |
| LCD1 | LM016L 16×2 LCD | Profile and status display | 3.3 V |
| D1, R1 | Red LED, 330 Ω | Status indication | P0.1 |
| M1 | MOTOR-DC | Load | 12 V via L293D |
| BAT1–3 | Batteries | 3.3 V, 5 V and 12 V supplies | — |
| S1 (optional) | Push-button + 10 kΩ pull-up | NEXT profile | 3.3 V |

**Table 4.2:** *Pin assignment*

| LPC2138 pin (no.) | Function | Connected to |
|---|---|---|
| P0.4 (27) | SCK0 | 25LC256 SCK (6) |
| P0.5 (29) | MISO0 | 25LC256 SO (2) |
| P0.6 (30) | MOSI0 | 25LC256 SI (5) |
| P0.7 (31) | GPIO chip select | 25LC256 CS (1) |
| P0.1 (21) | GPIO output | LED D1 through R1 |
| P0.8 (33) | Software PWM | L293D EN1 (1) |
| P0.12 (38), P0.13 (39) | GPIO outputs | L293D IN1 (2), IN2 (7) |
| P0.15 (45) | GPIO input (optional) | NEXT button to ground |
| P1.16 (16), P1.17 (12) | GPIO outputs | LCD RS (4), E (6) |
| P1.18–P1.21 (8, 4, 48, 44) | GPIO outputs | LCD D4–D7 (11–14) |

## 4.3 Power Supply

**Table 4.3:** *Supply connections*

| Supply | Connected to |
|---|---|
| BAT1 3.3 V | LPC2138 V3, V3A, VREF, VBAT; 25LC256 VCC, WP and HOLD; LCD VDD |
| BAT2 5 V | L293D VSS (pin 16, logic supply) |
| BAT3 12 V | L293D VS (pin 8, motor supply) |
| Ground | All negative terminals, LPC2138 VSS/VSSA, L293D GND, LCD VSS and VEE |

**Table 4.4:** *Proteus MOTOR-DC properties used*

| Property | Value | Note |
|---|---|---|
| Nominal voltage | 12 V | Matches BAT3 |
| Coil resistance / inductance | 12 Ω / 100 mH | Default model values |
| Zero-load speed | 1000 RPM | — |
| Load / max torque | 50 % | — |

| Property | Value | Note |
|---|---|---|
| Effective mass | 0.0005 | Reduced from 0.01 so that the speed settles within about 0.3 s |

### 4.4 Proteus Schematic

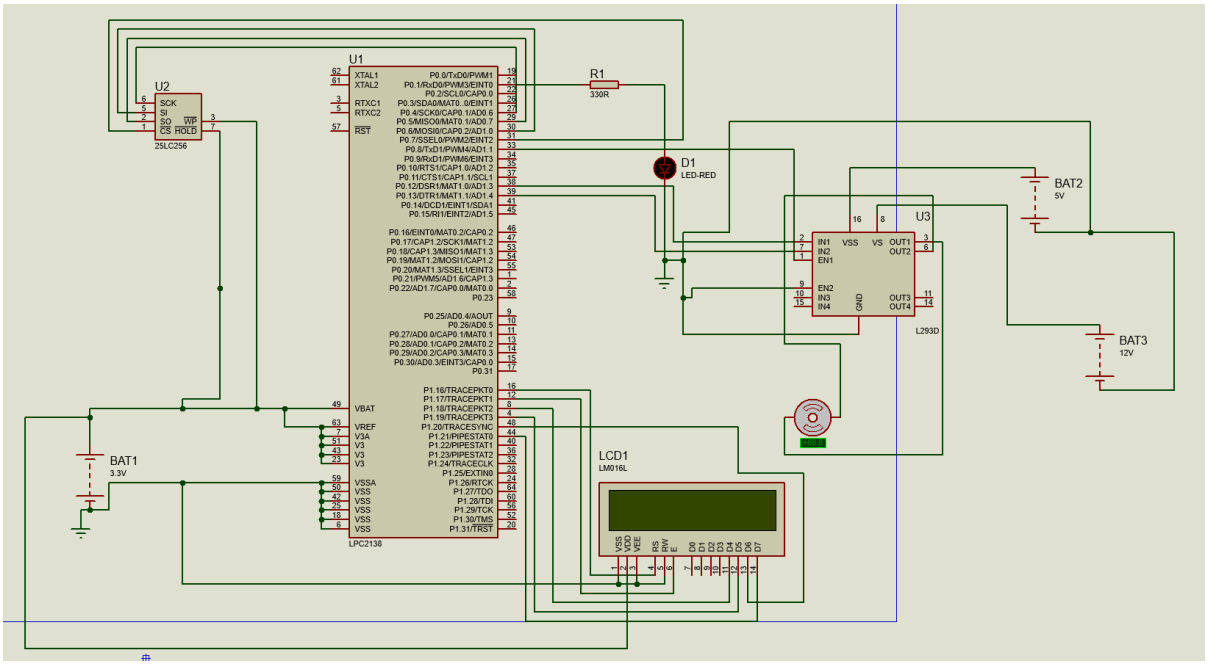



**Figure 4.2:** *Proteus schematic of the complete system*

## 5. MOTOR PROFILES AND EEPROM FORMAT

### 5.1 Daily-Life Motor Profiles

Six profiles are stored. Each imitates a real appliance and is run a set number of times before the controller moves to the next one. After the last profile the sequence starts again.

**Table 5.1:** *Motor profiles and their applications*

| No. | Name | Application | Behaviour | Repeats | Duration |
|---|---|---|---|---|---|
| 1 | DEMO | Test / demonstration | Forward 75 %, stop, reverse 40 %, stop | 1 | 9.6 s |
| 2 | WASH | Washing-machine wash | Agitation: forward 60 % 2 s, stop, reverse 60 % 2 s, stop | 3 | 18.6 s |
| 3 | SPIN | Washing-machine spin-dry | Gentle start at 40 %, ramp to 100 %, long coast-down | 1 | 15.0 s |
| 4 | GATE | Gate / garage door | Open (forward 80 % 4 s), wait 3 s, close (reverse 80 % 4 s) | 1 | 16.2 s |
| 5 | FAN | Natural-breeze fan | 30 % → 60 % → 90 % → 50 %, 2 s each, smooth changes | 2 | 24.0 s |
| 6 | MIXER | Blender pulse | Full-speed bursts of 0.7 s with 0.5 s pauses | 3 | 4.8 s |

**Table 5.2:** *Steps of each profile (direction, speed %, ramp ms, run ms)*

| Profile | Step 1 | Step 2 | Step 3 | Step 4 |
|---|---|---|---|---|
| DEMO | FWD 75, 500, 3000 | STOP, 300, 1000 | REV 40, 500, 3000 | STOP, 300, 1000 |

| Profile | Step 1 | Step 2 | Step 3 | Step 4 |
|---|---|---|---|---|
| WASH | FWD 60, 300, 2000 | STOP, 300, 500 | REV 60, 300, 2000 | STOP, 300, 500 |
| SPIN | FWD 40, 1000, 2000 | FWD 100, 2000, 5000 | STOP, 3000, 2000 | — |
| GATE | FWD 80, 800, 4000 | STOP, 800, 3000 | REV 80, 800, 4000 | STOP, 800, 2000 |
| FAN | FWD 30, 1000, 2000 | FWD 60, 1000, 2000 | FWD 90, 1000, 2000 | FWD 50, 1000, 2000 |
| MIXER | FWD 100, 200, 700 | STOP, 200, 500 | — | — |

For a STOP step, the ramp time is used to slow the motor down and the run time is the time the outputs stay off.

## 5.2 EEPROM Page Format

Profile k is stored in page k, at address k × 64. Keeping each profile in its own page means one profile can be rewritten in a single page write without touching the others, and a corrupted page only affects that profile.

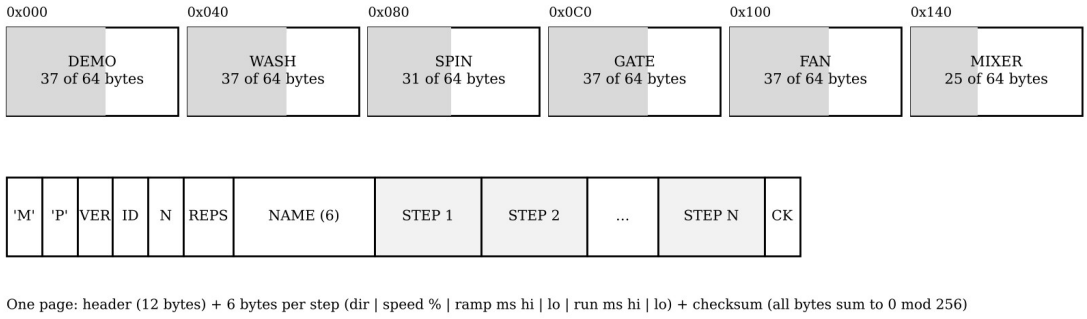



**Figure 5.1:** *EEPROM page map and the layout of one profile page*

**Table 5.3:** *Profile page byte layout*

| Offset | Field | Rule |
|---|---|---|
| 0–1 | Magic 'M', 'P' | Must be 0x4D, 0x50 |
| 2 | Format version | Must be 3 |
| 3 | Profile id | Must equal the page number k |
| 4 | Number of steps N | 1 to 8 |
| 5 | Repeats | 1 to 9 |
| 6–11 | Name | Up to 6 ASCII characters, padded with 0x00 |
| 12 + 6i | Step i | Direction (0 STOP, 1 FWD, 2 REV), speed (0–100 %), ramp ms, run ms (16-bit, big-endian) |
| 12 + 6N | Checksum | Chosen so that all bytes of the record sum to 0 (mod 256) |

At power-up each page is read and checked. If any test fails — a blank chip reads 0xFF, a page from an older format has the wrong version, or a byte has changed — the default profile for that page is written, read back and checked again.

# 6. WORKING PRINCIPLE AND DESIGN CALCULATIONS

## 6.1 Operating Sequence

1. The PLL is switched off so that the CPU runs at the 12 MHz crystal frequency, whatever the start-up file configured. Timer1 is started as a free-running 10 µs counter.
2. The LCD is initialised, the LED blinks three times and the six profiles are loaded from the EEPROM, rewriting any invalid page.
3. Each profile is run for its number of repeats. For each step the motor first ramps down if the direction changes, waits for the 200 ms dead time if it is reversing, sets IN1/IN2, ramps linearly to the new speed and holds it for the run time.
4. Every wait runs the `service()` routine, which keeps the PWM, the millisecond counter, the LED pattern, the button debounce and the LCD going.
5. When the last profile ends, or the NEXT button is pressed, the controller moves on to the next profile.

All step times are added to one absolute schedule on Timer1 rather than measured from 'now'. Small delays therefore never add up, and every profile takes exactly its designed time.

## 6.2 Design Calculations

**Table 6.1:** *Design calculations*

| Quantity | Calculation | Result |
|---|---|---|
| CPU clock | 12 MHz crystal, PLL disabled | CCLK = 12 MHz |
| Peripheral clock | CCLK / 4 (VPBDIV = 0) | PCLK = 3 MHz |
| Timer1 tick | (T1PR + 1) / PCLK = 30 / 3 MHz | 10 µs |
| PWM period | 1000 ticks × 10 µs | 10 ms (100 Hz) |
| PWM resolution | 10 ms / 100 | 1 % = 100 µs |
| SPI clock | PCLK / S0SPCCR = 3 MHz / 8 | 375 kHz, 21.3 µs per byte |
| Profile load time | 6 pages × 67 bytes × 21.3 µs | ≈ 8.6 ms |
| EEPROM programming (blank chip) | 6 page writes × ≤ 5 ms | ≤ 30 ms |
| LCD byte spacing | 5 ticks | 50 µs (controller needs 37 µs) |
| LCD screen refresh | 34 bytes × ≈ 50 µs, in the background | ≈ 2 ms |
| Ramp step (DEMO forward) | 500 ms / 75 steps | 6.67 ms per 1 % |
| Dead time | Minimum off time before reversing | 200 ms |
| Button debounce | 20 consecutive 1 ms samples | 20 ms |
| Full sequence | 9.6 + 18.6 + 15.0 + 16.2 + 24.0 + 4.8 | 88.2 s |
| Timer wrap | $2^{32}$ × 10 µs (wrap-safe comparisons) | ≈ 11.9 h |
| LED current | (3.3 V − 1.8 V) / 330 Ω | ≈ 4.5 mA |

## 6.3 LED Status Patterns

The LED pattern is chosen from what the motor is doing. Each pattern is a 1-second frame of ten 100 ms slots, so the patterns are easy to tell apart by eye (Figure 6.1).

**Table 6.2:** *LED patterns*

| Motor state | LED pattern |
|---|---|
| Start-up | 3 quick blinks (0.1 s ON, 0.1 s OFF) |
| Accelerating or decelerating | Fast blink, 5 Hz |
| Forward, holding speed | Slow blink (0.5 s ON / 0.5 s OFF) |
| Reverse, holding speed | Double flash every 1 s |
| Stopped, or dead time before reversing | Single short flash every 1 s |
| EEPROM or SPI error | Solid ON, motor kept off |

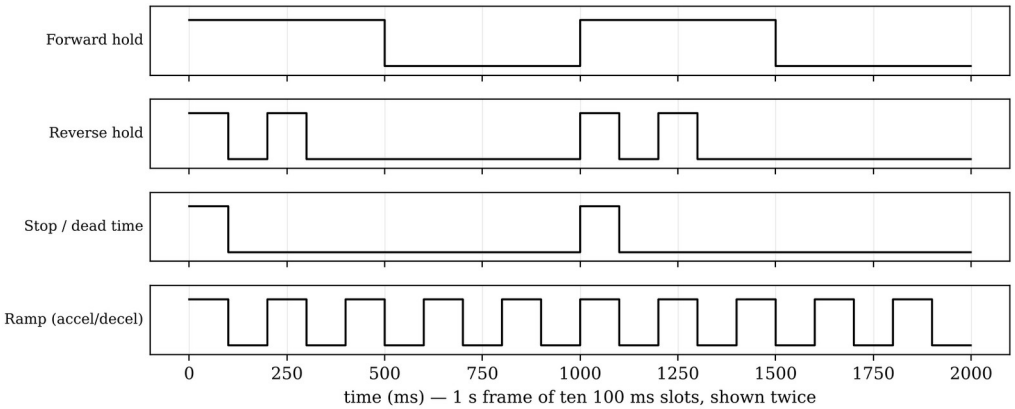



**Figure 6.1:** *LED patterns over two seconds*

### 6.4 LCD Screen Layout

Line 1 shows the profile name, the direction and the target speed. Line 2 shows the phase (ACCEL, HOLD, DECEL, DEAD or OFF), the step number and the repeat number. For example:

```
WASH   FWD  60%      <- profile, direction, target speed
HOLD  S1/4  R2/3     <- phase, step 1 of 4, repeat 2 of 3
```

## 7. SOFTWARE DESIGN

### 7.1 Build Process in Keil µVision

1. Create a new µVision project and select the device NXP LPC2138 (or LPC2148). Accept the default start-up file.
2. Add `main.c` to the source group.
3. In Options for Target: on the Output tab tick 'Create HEX File'; on the C/C++ tab set Optimization to Level 2.
4. Build the target (F7). The output window reports the code size and creates the .hex file.

The firmware switches the PLL off at the start of `main()`, so it runs correctly whatever clock settings the start-up file uses.

### 7.2 Running the Simulation

In Proteus, the .hex file is selected as the Program File of U1 and the Clock Frequency is set to 12 MHz, the value assumed by the firmware (`FOSC_HZ`). When the simulation starts, the LCD shows 'MOTOR PROFILES / Loading EEPROM', the LED blinks three times and the DEMO profile starts after about 0.7 s. The EEPROM contents can be viewed with Debug → SPI Memory, and the variables `g_profile`, `g_step`, `g_phase` and `g_speed` can be watched in the Watch Window.

## 7.3 Firmware Structure

**Table 7.1:** *Main firmware functions*

| Function(s) | Purpose |
|---|---|
| `service()` | Runs inside every wait: millisecond counter, software PWM on EN1, LED pattern, button debounce and one LCD byte |
| `wait_until()`, `wait_ms()` | Waits on absolute Timer1 times (no drift); return early if NEXT is pressed |
| `lcd_init()`, `lcd_show()`, `lcd_next()`, `lcd_status()` | LCD driver: a new screen is queued and sent one byte at a time in the background |
| `spi()`, `ee()`, `ee_write()` | SPI byte transfer with time-out; EEPROM command framing; page write with WEL check and WIP polling |
| `encode()`, `decode()`, `load()` | Profile page format, validation and automatic restore of defaults |
| `set_duty()`, `set_dir()`, `ramp_to()` | Motor outputs; IN1 and IN2 never high together; linear ramps of exact length |
| `run_step()`, `run_profile()`, `skip_profile()` | Step sequencing, dead time, repeats and NEXT handling |
| `main()` | Clock, pins and timer set-up, LCD start-up, EEPROM load and the profile loop |

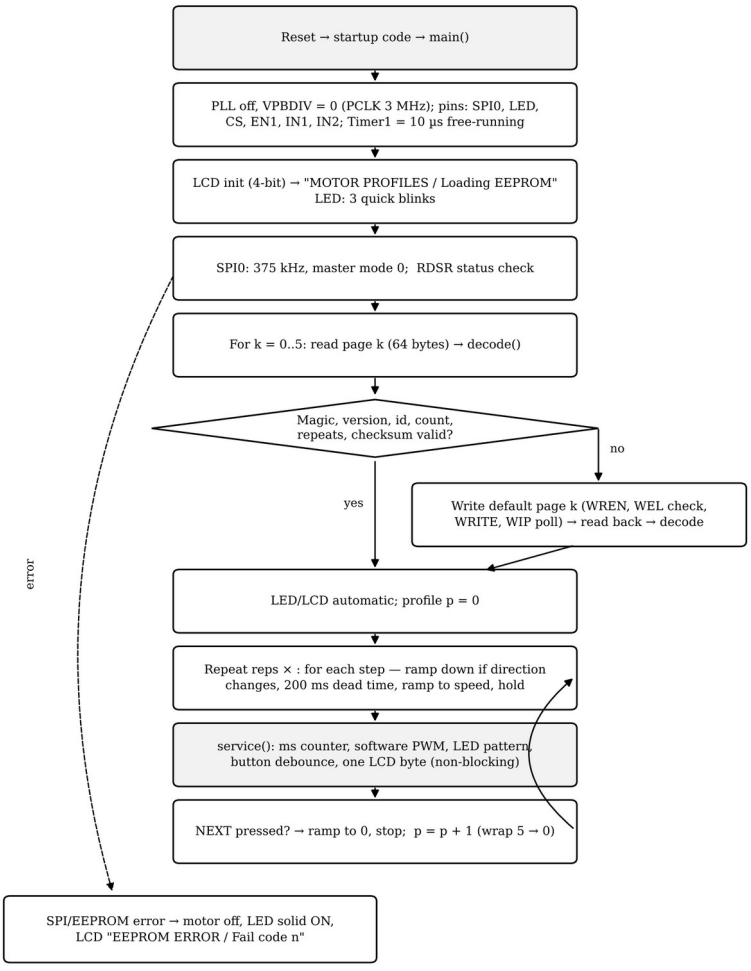



**Figure 7.1:** *Firmware flowchart*

### 7.4 Error Handling

Every SPI transfer and every EEPROM write has a time-out, so the program can never hang. If loading fails, the motor stays off, the LED stays on and the LCD shows the fault code.

**Table 7.2:** *Fault codes*

| Code | Meaning |
|---|---|
| 1 | SPI time-out (no transfer completed) |
| 2 | SPI mode fault or slave abort |
| 3 | No EEPROM: status register reads 0xFF |
| 4 | EEPROM write-protected (BP1:BP0 = 11) |
| 5 | Write enable did not set WEL |
| 6 | Write cycle did not finish (WIP stuck) |
| 9 | Profile still invalid after rewriting the defaults |

## 8. SOURCE CODE

The complete firmware is a single C file. Only short comments that explain non-obvious points are kept.

### 8.1 Motor-Profile Controller (main.c)

**Listing 8.1: main.c** *(574 lines)*

```
/* LPC2148 SPI EEPROM motor-profile controller
 * 12 MHz crystal, PLL off, PCLK 3 MHz
 * 25LC256: SCK P0.4, MISO P0.5, MOSI P0.6, CS P0.7
 * L293D: EN1 P0.8, IN1 P0.12, IN2 P0.13; LED P0.1
 * 16x2 LCD (4-bit): RS P1.16, E P1.17, D4-D7 P1.18-P1.21, RW to GND
 * Optional NEXT button: P0.15 to GND, 10k pull-up to 3.3 V
 * Six profiles, one per 64-byte EEPROM page, run in sequence.
 * All waits run service(); nothing blocks after start-up.
 */

#define FOSC_HZ         12000000U
#define EE_PAGE_SIZE    64U
#define DEAD_TIME_MS    200U
#define PROFILE_VERSION 3
#define SKIP_RAMP_MS    300U
#define DEBOUNCE_MS     20U

#define R(a) (*(volatile unsigned int *)(a))
#define PINSEL0 R(0xE002C000)
#define PINSEL2 R(0xE002C014)
#define IO0PIN  R(0xE0028000)
#define IO0SET  R(0xE0028004)
#define IO0DIR  R(0xE0028008)
#define IO0CLR  R(0xE002800C)
#define IO1SET  R(0xE0028014)
#define IO1DIR  R(0xE0028018)
#define IO1CLR  R(0xE002801C)
#define S0SPCR  R(0xE0020000)
#define S0SPSR  R(0xE0020004)
#define S0SPDR  R(0xE0020008)
#define S0SPCCR R(0xE002000C)
#define T1TCR   R(0xE0008004)
#define T1TC    R(0xE0008008)
#define T1PR    R(0xE000800C)
#define T1MCR   R(0xE0008014)
#define PLLCON  R(0xE01FC080)
#define PLLFEED R(0xE01FC08C)
#define PCONP   R(0xE01FC0C4)
#define VPBDIV  R(0xE01FC100)
#define SCS     R(0xE01FC1A0)

#define PCLK_HZ      (FOSC_HZ / 4U)
#define TICK_HZ      100000U            /* Timer1 tick = 10 us */
#define TICKS_PER_MS (TICK_HZ / 1000U)
#define PWM_PERIOD   1000U              /* 10 ms = 100 Hz */
#define TICKS_PER_PC (PWM_PERIOD / 100U)

#define LED (1U << 1)
```

```
#define CS  (1U << 7)
#define EN1 (1U << 8)
#define IN1 (1U << 12)
#define IN2 (1U << 13)
#define BTN (1U << 15)

#define LCD_RS    (1U << 16)
#define LCD_E     (1U << 17)
#define LCD_D4    18
#define LCD_PINS  (0x3FU << 16)
#define LCD_FRAME 34U                   /* 2 address commands + 32 characters */
#define LCD_GAP   5U                    /* 50 us between bytes (needs 37 us) */

#define PINSEL0_MASK 0xCF03FF0CU
#define PINSEL0_SPI  0x00001500U        /* P0.4-P0.6 = SPI0, P0.7 = GPIO */

#define EE_WRITE 0x02
#define EE_READ  0x03
#define EE_RDSR  0x05
#define EE_WREN  0x06
#define SR_WIP   0x01
#define SR_WEL   0x02
#define SR_BP    0x0C

/* LED patterns: 1 s frame, bit n = LED on during 100 ms slot n */
#define LED_FWD     0x01FU              /* slow blink */
#define LED_REV     0x005U              /* double flash */
#define LED_STOP    0x001U              /* single short flash */
#define LED_RAMP    0x155U              /* fast blink */
#define LED_SLOTS   10U
#define LED_SLOT_MS 100U

#define MAX_STEPS 8
#define NAME_LEN  6
#define HDR       (6 + NAME_LEN)        /* 'M' 'P' ver id n reps name */
#define NO_ADDR   (-1)

enum { STOP, FWD, REV };
enum { OK, E_SPI_TIMEOUT, E_SPI_FAULT, E_NO_EEPROM, E_PROTECTED, E_WEL, E_WRITE_TIMEOUT, E_PROFILE =
9 };
enum { PH_IDLE, PH_DECEL, PH_DEAD, PH_ACCEL, PH_HOLD };

typedef struct {
    unsigned char dir, speed;           /* speed in % */
    unsigned short ramp, run;           /* ms */
} step_t;

typedef struct {
    char name[NAME_LEN + 1];
    unsigned char reps, n;              /* repeats, steps */
    step_t s[MAX_STEPS];
} profile_t;

/* Defaults written to blank/invalid EEPROM pages */
static const profile_t defaults[] = {
    { "DEMO", 1, 4, {                   /* original test profile */
        { FWD, 75, 500, 3000 }, { STOP, 0, 300, 1000 },
        { REV, 40, 500, 3000 }, { STOP, 0, 300, 1000 } } },
    { "WASH", 3, 4, {                   /* washing machine agitation */
        { FWD, 60, 300, 2000 }, { STOP, 0, 300, 500 },
        { REV, 60, 300, 2000 }, { STOP, 0, 300, 500 } } },
    { "SPIN", 1, 3, {                   /* spin dry: slow start, full speed, long coast */
        { FWD, 40, 1000, 2000 }, { FWD, 100, 2000, 5000 },
        { STOP, 0, 3000, 2000 } } },
    { "GATE", 1, 4, {                   /* gate / garage door: open, wait, close */
        { FWD, 80, 800, 4000 }, { STOP, 0, 800, 3000 },
        { REV, 80, 800, 4000 }, { STOP, 0, 800, 2000 } } },
    { "FAN", 2, 4, {                    /* natural-breeze fan */
        { FWD, 30, 1000, 2000 }, { FWD, 60, 1000, 2000 },
        { FWD, 90, 1000, 2000 }, { FWD, 50, 1000, 2000 } } },
    { "MIXER", 3, 2, {                  /* blender pulse */
        { FWD, 100, 200, 700 }, { STOP, 0, 200, 500 } } }
};
#define NPROFILES (sizeof(defaults) / sizeof(defaults[0]))

static profile_t prof[NPROFILES];
static unsigned char page[EE_PAGE_SIZE];
static unsigned int duty, sched, pwm_start, ms_mark, last_driven, off_since, en1;
static unsigned int skip, btn_stable, btn_cnt;
static unsigned int led_auto, led_lit, led_ms, led_slot;
static unsigned int lcd_left, lcd_t;
static char lcd_buf[32];

volatile unsigned int g_fail_code, g_profile_source, g_profile, g_rep, g_step;
volatile unsigned int g_direction, g_speed, g_phase, g_cycles, g_time_ms, g_led_pattern;

/* ARM7 has no divide instruction */
static unsigned int udiv(unsigned int n, unsigned int d)
{
```

```
    unsigned int q = 0, bit = 1;
    if (d == 0) return 0;
    while (d < n && !(d & 0x80000000U)) { d <<= 1; bit <<= 1; }
    for (; bit; d >>= 1, bit >>= 1)
        if (n >= d) { n -= d; q |= bit; }
    return q;
}

/* ---------------- LCD ---------------- */

static void lcd_nibble(unsigned int rs, unsigned int v)
{
    volatile int i;
    IO1CLR = LCD_PINS;
    IO1SET = (rs ? LCD_RS : 0U) | ((v & 0xFU) << LCD_D4);
    IO1SET = LCD_E;
    for (i = 0; i < 3; i++) { }
    IO1CLR = LCD_E;
}

static void lcd_byte(unsigned int rs, unsigned int v)
{
    lcd_nibble(rs, v >> 4);
    lcd_nibble(rs, v);
}

/* Sends one byte of the pending frame; called from service() */
static void lcd_next(void)
{
    unsigned int p = LCD_FRAME - lcd_left--;
    if (p == 0) lcd_byte(0, 0x80);
    else if (p == 17) lcd_byte(0, 0xC0);
    else lcd_byte(1, (unsigned char)lcd_buf[p < 17 ? p - 1 : p - 2]);
    lcd_t = T1TC;
}

/* Queue a new screen; restarts any frame in progress */
static void lcd_show(const char *l1, const char *l2)
{
    unsigned int i;
    for (i = 0; i < 16; i++) { lcd_buf[i] = l1[i]; lcd_buf[i + 16] = l2[i]; }
    lcd_left = LCD_FRAME;
}

/* ---------------- Time base ---------------- */

static void led_update(void)
{
    unsigned int p, on;
    if (g_phase == PH_ACCEL || g_phase == PH_DECEL) p = LED_RAMP;
    else if (g_direction == FWD) p = LED_FWD;
    else if (g_direction == REV) p = LED_REV;
    else p = LED_STOP;
    if (p != g_led_pattern) { g_led_pattern = p; led_ms = 0; led_slot = 0; }
    on = (p >> led_slot) & 1U;
    if (on != led_lit) {
        if (on) IO0SET = LED; else IO0CLR = LED;
        led_lit = on;
    }
}

/* NEXT button: debounced press sets skip */
static void button(void)
{
    unsigned int b = (IO0PIN & BTN) ? 1U : 0U;
    if (b == btn_stable) btn_cnt = 0;
    else if (++btn_cnt >= DEBOUNCE_MS) {
        btn_stable = b;
        btn_cnt = 0;
        if (!b) skip = 1;
    }
}

/* Runs in every wait loop: ms counter, software PWM, LED, button, LCD */
static void service(void)
{
    unsigned int t = T1TC, on;
    while (t - ms_mark >= TICKS_PER_MS) {
        ms_mark += TICKS_PER_MS;
        g_time_ms++;
        if (++led_ms == LED_SLOT_MS) { led_ms = 0; if (++led_slot == LED_SLOTS) led_slot = 0; }
        if (led_auto) button();
    }
    while (t - pwm_start >= PWM_PERIOD) pwm_start += PWM_PERIOD;
    on = t - pwm_start < duty * TICKS_PER_PC;
    if (on != en1) {
        if (on) IO0SET = EN1; else IO0CLR = EN1;
        en1 = on;
    }
```

```
    if (led_auto) led_update();
    if (lcd_left && t - lcd_t >= LCD_GAP) lcd_next();
}

/* Returns 0 if a button press cut the wait short */
static unsigned int wait_until(unsigned int deadline)
{
    while ((int)(T1TC - deadline) < 0) {
        if (skip) return 0;
        service();
    }
    return 1;
}

/* Absolute scheduling: no drift */
static unsigned int wait_ms(unsigned int ms)
{
    sched += ms * TICKS_PER_MS;
    return wait_until(sched);
}

/* Start-up only: relative wait */
static void pause_ms(unsigned int ms)
{
    unsigned int t = T1TC + ms * TICKS_PER_MS;
    while ((int)(T1TC - t) < 0) service();
}

static void lcd_init(void)
{
    PINSEL2 &= ~(1U << 3);              /* P1.16-P1.25 as GPIO */
    IO1CLR = LCD_PINS;
    IO1DIR |= LCD_PINS;
    pause_ms(40);
    lcd_nibble(0, 3); pause_ms(5);
    lcd_nibble(0, 3); pause_ms(1);
    lcd_nibble(0, 3); pause_ms(1);
    lcd_nibble(0, 2); pause_ms(1);      /* 4-bit mode */
    lcd_byte(0, 0x28); pause_ms(1);     /* 2 lines, 5x8 */
    lcd_byte(0, 0x0C); pause_ms(1);     /* display on, no cursor */
    lcd_byte(0, 0x06); pause_ms(1);
    lcd_byte(0, 0x01); pause_ms(2);     /* clear */
}

/* Right-aligned decimal in w characters */
static void put_num(char *p, unsigned int w, unsigned int v)
{
    do {
        unsigned int q = udiv(v, 10);
        p[--w] = (char)('0' + v - q * 10);
        v = q;
    } while (v && w);
}

static void copy(char *d, const char *s)
{
    while (*s) *d++ = *s++;
}

/* Line 1: profile, direction, target %. Line 2: phase, step, repeat */
static void lcd_status(unsigned int target)
{
    static const char *const dir_txt[] = { "STOP", "FWD", "REV" };
    static const char *const ph_txt[] = { "IDLE", "DECEL", "DEAD", "ACCEL", "HOLD" };
    const profile_t *p = &prof[g_profile];
    char a[17] = "                ", b[17] = "      S /   R / ";
    copy(a, p->name);
    copy(a + 7, dir_txt[g_direction]);
    put_num(a + 11, 3, target);
    a[14] = '%';
    copy(b, g_direction == STOP && g_phase == PH_HOLD ? "OFF" : ph_txt[g_phase]);
    put_num(b + 7, 1, g_step + 1);
    put_num(b + 9, 1, p->n);
    put_num(b + 13, 1, g_rep + 1);
    put_num(b + 15, 1, p->reps);
    lcd_show(a, b);
}

static void blink(void)
{
    IO0SET = LED; wait_ms(100);
    IO0CLR = LED; wait_ms(100);
}

/* ---------------- SPI0 and 25LC256 ---------------- */

static void cs(int high)
{
    volatile int i;
```

```
315      for (i = 0; i < 10; i++) { }
316      if (high) IO0SET = CS; else IO0CLR = CS;
317      for (i = 0; i < 10; i++) { }
318  }
319
320  /* SPIF cleared by reading S0SPSR then S0SPDR */
321  static unsigned int spi(unsigned char tx, unsigned char *rx)
322  {
323      unsigned int n = 100000, s;
324      S0SPDR = tx;
325      while (!((s = S0SPSR) & 0x80)) {
326          if (s & 0x18) return E_SPI_FAULT;
327          if (--n == 0) return E_SPI_TIMEOUT;
328      }
329      s = S0SPDR;
330      if (rx) *rx = (unsigned char)s;
331      return OK;
332  }
333
334  static unsigned int ee(unsigned char cmd, int addr, const unsigned char *tx, unsigned char *rx,
unsigned int n)
335  {
336      unsigned int e, i;
337      cs(0);
338      e = spi(cmd, 0);
339      if (addr != NO_ADDR) {
340          if (!e) e = spi((unsigned char)((addr >> 8) & 0xFF), 0);
341          if (!e) e = spi((unsigned char)(addr & 0xFF), 0);
342      }
343      for (i = 0; i < n && !e; i++) e = spi(tx ? tx[i] : 0, rx ? &rx[i] : 0);
344      cs(1);
345      return e;
346  }
347
348  /* Page-aware write: WREN, check WEL, WRITE, poll WIP */
349  static unsigned int ee_write(unsigned int addr, const unsigned char *buf, unsigned int len)
350  {
351      unsigned int e, n, polls;
352      unsigned char st = 0;
353      for (; len; addr += n, buf += n, len -= n) {
354          n = EE_PAGE_SIZE - (addr % EE_PAGE_SIZE);
355          if (n > len) n = len;
356          if ((e = ee(EE_WREN, NO_ADDR, 0, 0, 0)) || (e = ee(EE_RDSR, NO_ADDR, 0, &st, 1))) return e;
357          if (!(st & SR_WEL)) return E_WEL;
358          if ((e = ee(EE_WRITE, (int)addr, buf, 0, n))) return e;
359          for (polls = 0; ; polls++) {
360              if (polls == 20000) return E_WRITE_TIMEOUT;
361              if ((e = ee(EE_RDSR, NO_ADDR, 0, &st, 1))) return e;
362              if (!(st & SR_WIP)) break;
363          }
364      }
365      return OK;
366  }
367
368  /* ---------------- Profile ---------------- */
369
370  static unsigned char sum(const unsigned char *b, unsigned int n)
371  {
372      unsigned char s = 0;
373      while (n--) s += *b++;
374      return s;
375  }
376
377  /* Page k: 'M' 'P' ver k n reps name[6], 6 bytes per step, checksum */
378  static unsigned int encode(unsigned int k, unsigned char *b)
379  {
380      const profile_t *p = &defaults[k];
381      unsigned int i, n = HDR;
382      b[0] = 'M'; b[1] = 'P'; b[2] = PROFILE_VERSION; b[3] = (unsigned char)k;
383      b[4] = p->n; b[5] = p->reps;
384      for (i = 0; i < NAME_LEN; i++) b[6 + i] = (unsigned char)p->name[i];
385      for (i = 0; i < p->n; i++) {
386          const step_t *s = &p->s[i];
387          b[n++] = s->dir;
388          b[n++] = s->speed;
389          b[n++] = (unsigned char)(s->ramp >> 8); b[n++] = (unsigned char)s->ramp;
390          b[n++] = (unsigned char)(s->run >> 8);  b[n++] = (unsigned char)s->run;
391      }
392      b[n] = (unsigned char)(0 - sum(b, n));
393      return n + 1;
394  }
395
396  static unsigned int decode(unsigned int k, const unsigned char *b)
397  {
398      profile_t *p = &prof[k];
399      unsigned int i, n = b[4];
400      if (b[0] != 'M' || b[1] != 'P' || b[2] != PROFILE_VERSION || b[3] != k) return 0;
401      if (n == 0 || n > MAX_STEPS || b[5] == 0 || b[5] > 9 || sum(b, HDR + n * 6 + 1) != 0) return 0;
402      for (i = 0; i < n; i++) {
```

```
403          const unsigned char *q = b + HDR + i * 6;
404          if (q[0] > REV || q[1] > 100) return 0;
405          p->s[i].dir = q[0];
406          p->s[i].speed = q[1];
407          p->s[i].ramp = (unsigned short)(q[2] << 8 | q[3]);
408          p->s[i].run = (unsigned short)(q[4] << 8 | q[5]);
409      }
410      for (i = 0; i < NAME_LEN; i++) p->name[i] = (char)b[6 + i];
411      p->name[NAME_LEN] = 0;
412      p->n = (unsigned char)n;
413      p->reps = b[5];
414      return 1;
415  }
416
417  /* Each blank or invalid page gets its default and is read back */
418  static unsigned int load(void)
419  {
420      unsigned char st = 0;
421      unsigned int e, k, addr;
422      if ((e = ee(EE_RDSR, NO_ADDR, 0, &st, 1))) return e;
423      if (st == 0xFF) return E_NO_EEPROM;
424      if ((st & SR_BP) == SR_BP) return E_PROTECTED;
425      g_profile_source = 1;
426      for (k = 0; k < NPROFILES; k++) {
427          addr = k * EE_PAGE_SIZE;
428          if ((e = ee(EE_READ, (int)addr, 0, page, EE_PAGE_SIZE))) return e;
429          if (decode(k, page)) continue;
430          if ((e = ee_write(addr, page, encode(k, page))) || (e = ee(EE_READ, (int)addr, 0, page,
EE_PAGE_SIZE)))
431              return e;
432          if (!decode(k, page)) return E_PROFILE;
433          g_profile_source = 2;
434      }
435      return OK;
436  }
437
438  /* ---------------- Motor ---------------- */
439
440  static void set_duty(unsigned int p)
441  {
442      duty = p > 100 ? 100 : p;
443      g_speed = duty;
444      service();
445  }
446
447  /* IN1 and IN2 never high together */
448  static void set_dir(unsigned int d)
449  {
450      IO0CLR = IN1 | IN2;
451      if (d == FWD) IO0SET = IN1;
452      else if (d == REV) IO0SET = IN2;
453      if (d == STOP && g_direction != STOP) { last_driven = g_direction; off_since = sched; }
454      g_direction = d;
455  }
456
457  /* Linear ramp, exactly ms long; returns 0 if skipped */
458  static unsigned int ramp_to(unsigned int target, unsigned int ms)
459  {
460      unsigned int from = duty, n, k, base = sched, total = ms * TICKS_PER_MS;
461      n = from > target ? from - target : target - from;
462      if (n == 0) return 1;
463      for (k = 1; k <= n; k++) {
464          if (!wait_until(base + udiv(total * k, n))) return 0;
465          set_duty(from < target ? from + k : from - k);
466      }
467      sched = base + total;
468      return 1;
469  }
470
471  static void run_step(const step_t *s)
472  {
473      unsigned int ready;
474
475      if (g_direction != STOP && s->dir != g_direction) {
476          g_phase = PH_DECEL;
477          lcd_status(0);
478          if (!ramp_to(0, s->ramp)) return;
479          set_dir(STOP);
480      }
481      if (s->dir != STOP) {
482          if (g_direction != s->dir) {
483              ready = off_since + DEAD_TIME_MS * TICKS_PER_MS;
484              if (last_driven != STOP && last_driven != s->dir && (int)(ready - sched) > 0) {
485                  g_phase = PH_DEAD;
486                  lcd_status(0);
487                  sched = ready;
488                  if (!wait_until(sched)) return;
489              }
490              set_dir(s->dir);
```

```
        }
        g_phase = PH_ACCEL;
        lcd_status(s->speed);
        if (!ramp_to(s->speed, s->ramp)) return;
    }
    g_phase = PH_HOLD;
    lcd_status(duty);
    wait_ms(s->run);
}

static void run_profile(const profile_t *p)
{
    for (g_rep = 0; g_rep < p->reps && !skip; g_rep++)
        for (g_step = 0; g_step < p->n; g_step++) {
            run_step(&p->s[g_step]);
            if (skip) return;
        }
}

/* Button pressed: ramp down, stop, then the next profile starts */
static void skip_profile(void)
{
    skip = 0;
    sched = T1TC;
    g_phase = PH_DECEL;
    lcd_status(0);
    ramp_to(0, SKIP_RAMP_MS);
    set_duty(0);
    sched = T1TC;
    set_dir(STOP);
}

int main(void)
{
    unsigned int i;

    /* PLL off in case startup code enabled it */
    PLLCON = 0;
    PLLFEED = 0xAA;
    PLLFEED = 0x55;
    VPBDIV = 0;
    SCS = 0;

    PINSEL0 = (PINSEL0 & ~PINSEL0_MASK) | PINSEL0_SPI;
    IO0SET = CS;
    IO0CLR = LED | EN1 | IN1 | IN2;
    IO0DIR |= LED | CS | EN1 | IN1 | IN2;
    PCONP |= (1U << 2) | (1U << 8);      /* Timer1, SPI0 */

    T1TCR = 2;
    T1PR = PCLK_HZ / TICK_HZ - 1;        /* 29 -> 10 us */
    T1MCR = 0;
    T1TCR = 1;

    pwm_start = ms_mark = T1TC;
    last_driven = STOP;
    set_dir(STOP);
    lcd_init();
    lcd_show("MOTOR PROFILES  ", "Loading EEPROM  ");

    sched = T1TC;
    for (i = 0; i < 3; i++) blink();

    S0SPCCR = 8;                         /* 375 kHz */
    S0SPCR = 0x20;                       /* master, mode 0 */
    g_fail_code = load();

    if (g_fail_code) {                   /* LED solid on, motor off */
        char e[17] = "Fail code       ";
        put_num(e + 10, 1, g_fail_code);
        lcd_show("EEPROM ERROR    ", e);
        IO0SET = LED;
        for (;;) service();
    }

    btn_stable = (IO0PIN & BTN) ? 1U : 0U;
    led_auto = 1;
    sched = T1TC;
    for (;;) {
        run_profile(&prof[g_profile]);
        if (skip) skip_profile();
        if (++g_profile == NPROFILES) { g_profile = 0; g_cycles++; }
    }
}
```

# 9. RESULTS AND DISCUSSION

## 9.1 Build and EEPROM Programming

The program compiled in Keil µVision with 0 errors (8 compiler warnings), giving 6488 bytes of code (Figure 9.1).

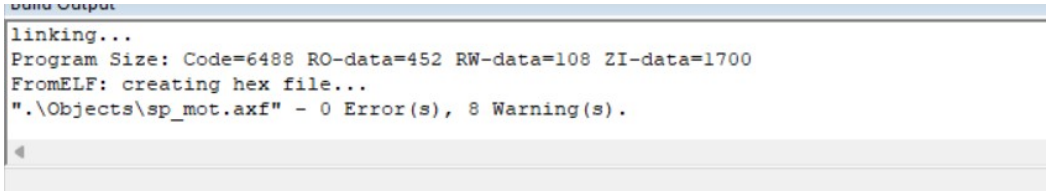



**Figure 9.1:** *Keil µVision build output*

On the first run the EEPROM was blank, so the firmware wrote the six default profiles. Figure 9.2 shows the 25LC256 memory in Proteus. The four pages visible (DEMO at 0x000, WASH at 0x040, SPIN at 0x080 and GATE at 0x0C0) match, byte for byte, the records produced by the firmware's encoder, including the checksums 0x20, 0x5E, 0xC4 and 0x10. For example, the DEMO page begins `4D 50 03 00 04 01 44 45 4D 4F`: 'MP', version 3, id 0, 4 steps, 1 repeat, 'DEMO'.

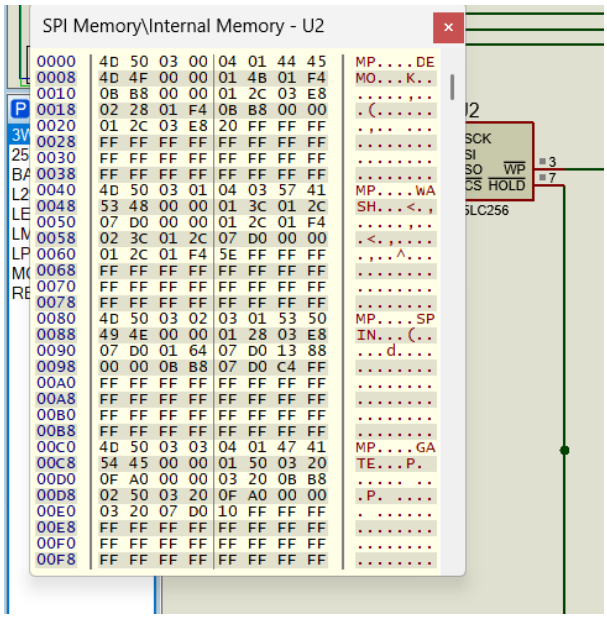
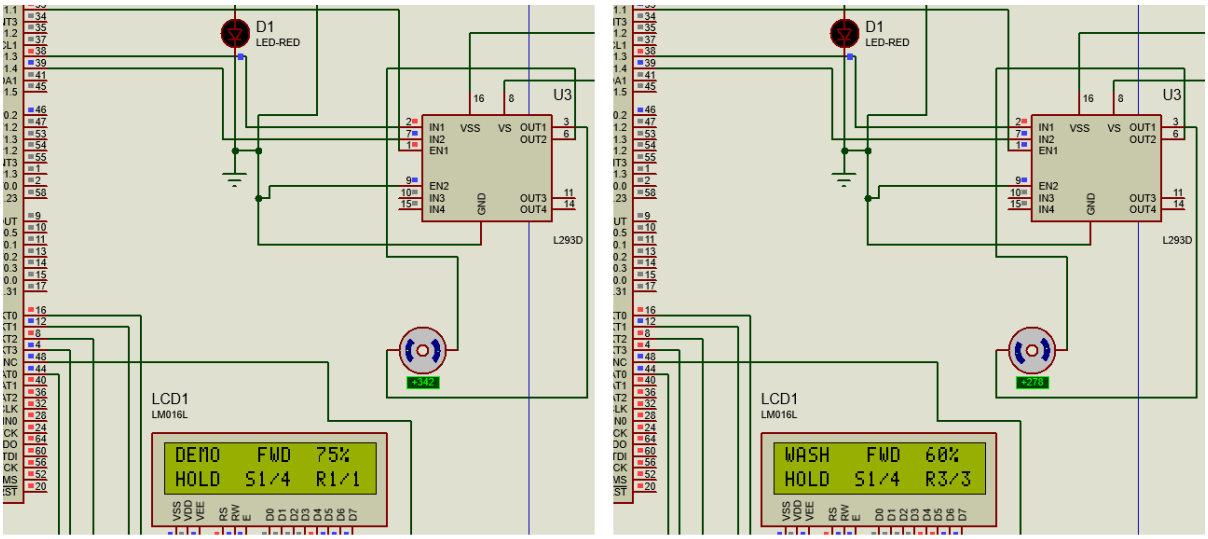



**Figure 9.2:** *25LC256 contents after the first run (Debug → SPI Memory)*

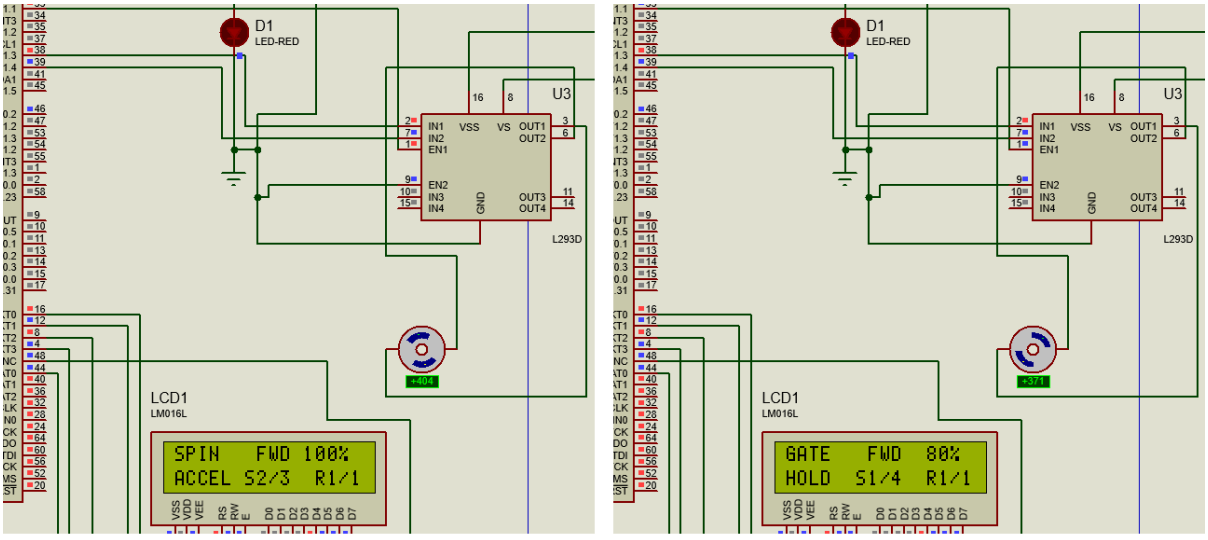
## 9.2 Profile Execution in Proteus

Figures 9.3 to 9.5 show the system running each profile. In every case the LCD, the L293D pin states and the motor speed agree: forward steps give a positive speed, the motor is driven only while IN1 or IN2 is high, and the LED shows the expected pattern.




(a) DEMO, forward hold (b) WASH, repeat 3 of 3

**Figure 9.3:** *DEMO at 3.4 s (+342 RPM) and WASH at 29.7 s (+278 RPM)*




(a) SPIN, ramp to 100 % (b) GATE, opening

**Figure 9.4:** *SPIN at 40.25 s (+404 RPM, rising) and GATE at 56.1 s (+371 RPM)*

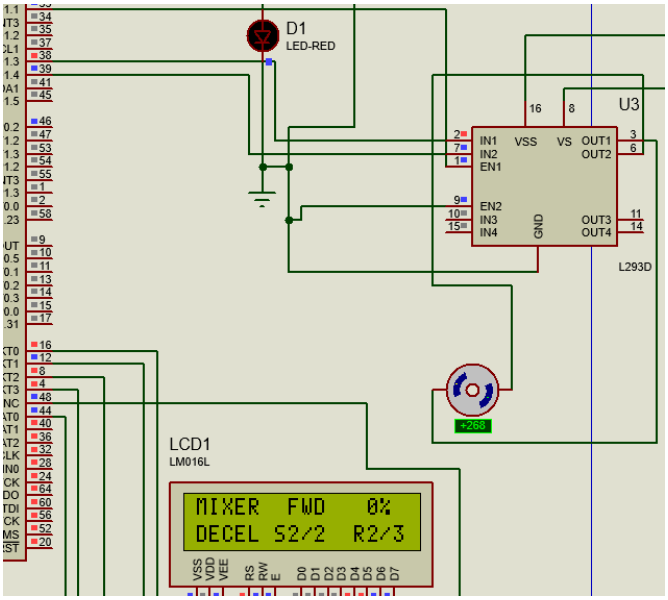



**Figure 9.5:** *MIXER at 104.0 s, slowing down after a pulse (+268 RPM)*

**Table 9.1:** *Motor speed observed in Proteus*

| Profile and step | Commanded duty | Motor speed |
|---|---|---|
| DEMO, forward hold | 75 % | +342 RPM |
| WASH, forward hold | 60 % | +278 RPM |
| GATE, forward hold (opening) | 80 % | +371 RPM |
| SPIN, ramp to 100 % | rising | +404 RPM and rising |
| MIXER, slowing after a 100 % pulse | falling | +268 RPM |

The holding speeds rise with the duty cycle (278, 342 and 371 RPM at 60, 75 and 80 %), as expected for PWM speed control.

## 9.3 LED and LCD Status over the Full Sequence

Table 9.2 lists what the LED and LCD show at each point of the full 88.2 s sequence. The times are measured from reset with the 12 MHz clock and were confirmed in the emulator (Section 9.4). Rows marked 'repeat' show the same sequence again with the next repeat number.

**Table 9.2:** *LED and LCD status at different times (full sequence)*

| Motor profile (time) | LED status | LCD display |
|---|---|---|
| Start-up<br>0 – 0.7 s | 3 quick blinks | `MOTOR PROFILES`<br>`Loading EEPROM` |
| DEMO<br>0.7 – 1.2 s | Fast blink, 5 Hz | `DEMO  FWD  75%`<br>`ACCEL S1/4  R1/1` |
| DEMO<br>1.2 – 4.2 s | Slow blink (0.5 s ON / 0.5 s OFF) | `DEMO  FWD  75%`<br>`HOLD  S1/4  R1/1` |
| DEMO<br>4.2 – 4.5 s | Fast blink, 5 Hz | `DEMO  FWD  0%`<br>`DECEL S2/4  R1/1` |
| DEMO<br>4.5 – 5.5 s | Single short flash every 1 s | `DEMO  STOP  0%`<br>`OFF  S2/4  R1/1` |
| DEMO<br>5.5 – 6.0 s | Fast blink, 5 Hz | `DEMO  REV  40%`<br>`ACCEL S3/4  R1/1` |
| DEMO<br>6.0 – 9.0 s | Double flash every 1 s | `DEMO  REV  40%`<br>`HOLD  S3/4  R1/1` |
| DEMO<br>9.0 – 9.3 s | Fast blink, 5 Hz | `DEMO  REV  0%`<br>`DECEL S4/4  R1/1` |
| DEMO<br>9.3 – 10.3 s | Single short flash every 1 s | `DEMO  STOP  0%`<br>`OFF  S4/4  R1/1` |
| WASH<br>10.3 – 10.6 s | Fast blink, 5 Hz | `WASH  FWD  60%`<br>`ACCEL S1/4  R1/3` |
| WASH<br>10.6 – 12.6 s | Slow blink (0.5 s ON / 0.5 s OFF) | `WASH  FWD  60%`<br>`HOLD  S1/4  R1/3` |
| WASH<br>12.6 – 12.9 s | Fast blink, 5 Hz | `WASH  FWD  0%`<br>`DECEL S2/4  R1/3` |
| WASH<br>12.9 – 13.4 s | Single short flash every 1 s | `WASH  STOP  0%`<br>`OFF  S2/4  R1/3` |
| WASH<br>13.4 – 13.7 s | Fast blink, 5 Hz | `WASH  REV  60%`<br>`ACCEL S3/4  R1/3` |
| WASH<br>13.7 – 15.7 s | Double flash every 1 s | `WASH  REV  60%`<br>`HOLD  S3/4  R1/3` |

| Motor profile (time) | LED status | LCD display |
|---|---|---|
| WASH<br>15.7 – 16.0 s | Fast blink, 5 Hz | `WASH  REV  0%`<br>`DECEL S4/4  R1/3` |
| WASH<br>16.0 – 16.5 s | Single short flash every 1 s | `WASH  STOP  0%`<br>`OFF  S4/4  R1/3` |
| WASH (repeat 2)<br>16.5 – 22.7 s | Same pattern as 10.3 – 16.5 s | Same, with `R2/3` |
| WASH (repeat 3)<br>22.7 – 28.9 s | Same pattern as 10.3 – 16.5 s | Same, with `R3/3` |
| SPIN<br>28.9 – 29.9 s | Fast blink, 5 Hz | `SPIN  FWD  40%`<br>`ACCEL S1/3  R1/1` |
| SPIN<br>29.9 – 31.9 s | Slow blink (0.5 s ON / 0.5 s OFF) | `SPIN  FWD  40%`<br>`HOLD  S1/3  R1/1` |
| SPIN<br>31.9 – 33.9 s | Fast blink, 5 Hz | `SPIN  FWD 100%`<br>`ACCEL S2/3  R1/1` |
| SPIN<br>33.9 – 38.9 s | Slow blink (0.5 s ON / 0.5 s OFF) | `SPIN  FWD 100%`<br>`HOLD  S2/3  R1/1` |
| SPIN<br>38.9 – 41.9 s | Fast blink, 5 Hz | `SPIN  FWD  0%`<br>`DECEL S3/3  R1/1` |
| SPIN<br>41.9 – 43.9 s | Single short flash every 1 s | `SPIN  STOP  0%`<br>`OFF  S3/3  R1/1` |
| GATE<br>43.9 – 44.7 s | Fast blink, 5 Hz | `GATE  FWD  80%`<br>`ACCEL S1/4  R1/1` |
| GATE<br>44.7 – 48.7 s | Slow blink (0.5 s ON / 0.5 s OFF) | `GATE  FWD  80%`<br>`HOLD  S1/4  R1/1` |
| GATE<br>48.7 – 49.5 s | Fast blink, 5 Hz | `GATE  FWD  0%`<br>`DECEL S2/4  R1/1` |
| GATE<br>49.5 – 52.5 s | Single short flash every 1 s | `GATE  STOP  0%`<br>`OFF  S2/4  R1/1` |
| GATE<br>52.5 – 53.3 s | Fast blink, 5 Hz | `GATE  REV  80%`<br>`ACCEL S3/4  R1/1` |
| GATE<br>53.3 – 57.3 s | Double flash every 1 s | `GATE  REV  80%`<br>`HOLD  S3/4  R1/1` |
| GATE<br>57.3 – 58.1 s | Fast blink, 5 Hz | `GATE  REV  0%`<br>`DECEL S4/4  R1/1` |
| GATE<br>58.1 – 60.1 s | Single short flash every 1 s | `GATE  STOP  0%`<br>`OFF  S4/4  R1/1` |
| FAN<br>60.1 – 61.1 s | Fast blink, 5 Hz | `FAN  FWD  30%`<br>`ACCEL S1/4  R1/2` |
| FAN<br>61.1 – 63.1 s | Slow blink (0.5 s ON / 0.5 s OFF) | `FAN  FWD  30%`<br>`HOLD  S1/4  R1/2` |
| FAN<br>63.1 – 64.1 s | Fast blink, 5 Hz | `FAN  FWD  60%`<br>`ACCEL S2/4  R1/2` |
| FAN<br>64.1 – 66.1 s | Slow blink (0.5 s ON / 0.5 s OFF) | `FAN  FWD  60%`<br>`HOLD  S2/4  R1/2` |
| FAN<br>66.1 – 67.1 s | Fast blink, 5 Hz | `FAN  FWD  90%`<br>`ACCEL S3/4  R1/2` |

| Motor profile (time) | LED status | LCD display |
|---|---|---|
| FAN<br>67.1 – 69.1 s | Slow blink (0.5 s ON / 0.5 s OFF) | `FAN  FWD 90%`<br>`HOLD S3/4 R1/2` |
| FAN<br>69.1 – 70.1 s | Fast blink, 5 Hz | `FAN  FWD 50%`<br>`ACCEL S4/4 R1/2` |
| FAN<br>70.1 – 72.1 s | Slow blink (0.5 s ON / 0.5 s OFF) | `FAN  FWD 50%`<br>`HOLD S4/4 R1/2` |
| FAN (repeat 2)<br>72.1 – 84.1 s | Same pattern as 60.1 – 72.1 s | Same, with `R2/2` |
| MIXER<br>84.1 – 84.3 s | Fast blink, 5 Hz | `MIXER FWD 100%`<br>`ACCEL S1/2 R1/3` |
| MIXER<br>84.3 – 85.0 s | Slow blink (0.5 s ON / 0.5 s OFF) | `MIXER FWD 100%`<br>`HOLD S1/2 R1/3` |
| MIXER<br>85.0 – 85.2 s | Fast blink, 5 Hz | `MIXER FWD 0%`<br>`DECEL S2/2 R1/3` |
| MIXER<br>85.2 – 85.7 s | Single short flash every 1 s | `MIXER STOP 0%`<br>`OFF S2/2 R1/3` |
| MIXER (repeats 2, 3)<br>85.7 – 88.9 s | Same pattern as 84.1 – 85.7 s | Same, with `R2/3`, then `R3/3` |
| DEMO (next cycle)<br>88.9 → s | Sequence starts again | `DEMO  FWD 75%`<br>`ACCEL S1/4 R1/1` |

## 9.4 Timing Verification

The firmware was also run in an instruction-level emulator with models of the peripherals. Figure 9.6 shows the measured drive signal over the full sequence. The measured results are summarised in Table 9.3.

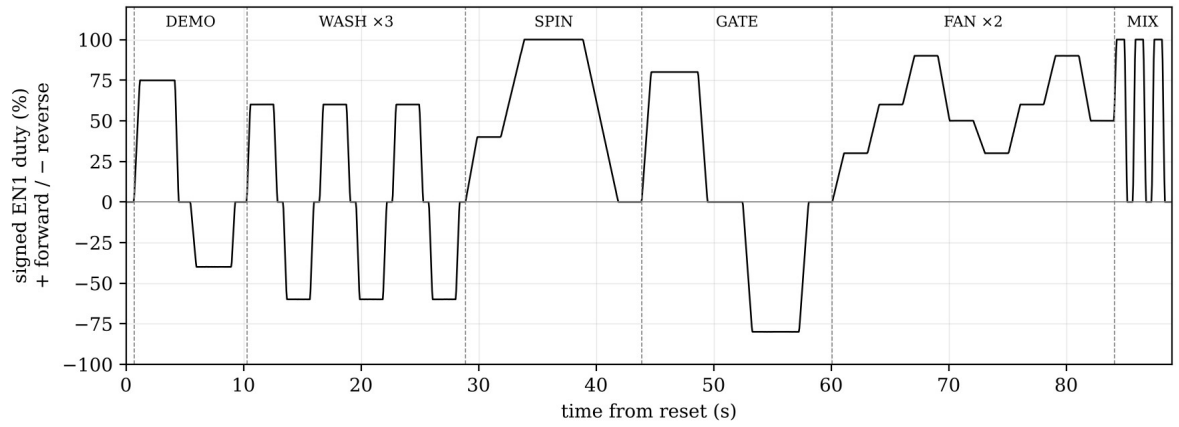



**Figure 9.6:** *Measured motor drive over the full 88.2 s sequence (emulation)*

**Table 9.3:** *Timing measured in emulation*

| Check | Result |
|---|---|
| Sequence period | 88.2 s, returning to DEMO at 88.9 s after reset |
| Stop intervals before each reversal | 0.5 – 3.0 s as designed; never below the 200 ms dead time |
| IN1 and IN2 high together | Never |
| PWM pulse width at 75 % / 40 % | 7.50 ms / 4.00 ms (±25 µs), period 10.00 ms |
| LCD writes while the controller was busy | None |

| Check | Result |
|---|---|
| NEXT button | Each press skipped exactly one profile; 3–4 ms contact bounce was ignored |
| Fault tests (EEPROM unpowered, SO stuck low, no SCK) | Codes 3, 5 and 1 shown on the LCD; LED on; motor off |

In the recorded Proteus run, every event happened 1.2 times later than the times in Table 9.2. For example, MIXER repeat 2 started slowing at 104.0 s instead of 86.6 s, and GATE was opening at 56.1 s instead of 44.7–48.7 s. Because the ratio is the same throughout the run, this is a constant clock difference, not drift. It matches the microcontroller model running at 10 MHz while the firmware assumes 12 MHz, so the Clock Frequency property of U1 should be checked and set to 12 MHz.

### 9.5 Problems Faced and Solutions

**Table 9.4:** *Main problems and how they were solved*

| Problem | Solution |
|---|---|
| The motor speed lagged several seconds behind the LCD; it had not reversed before the next step began | The motor model's effective mass (0.01) gave a 4 s response, longer than the 3 s holds. Reducing it to 0.0005 made the speed settle in about 0.3 s |
| Each LCD update paused the program for about 5 ms and delayed motor events | The LCD driver was made non-blocking: one byte is sent per `service()` call, 50 µs apart |
| Ramps and stops in an earlier version were 5–20 % off and the cycle drifted | All events were moved onto one absolute Timer1 schedule; the cycle is now exact |
| Keil's start-up file enables the PLL (60 MHz), which would make every time 5 times too short | `main()` switches the PLL off before setting up the timers |
| Keil licence errors with a separately installed ARM Compiler 5 | Reinstalled Keil MDK with legacy ARM7 support |
| Button contact bounce could skip several profiles | 20 ms debounce; only a press (falling edge) is acted on |

### 9.6 Discussion

The results show that one firmware image can behave like several different appliances simply by changing data in the EEPROM. The page-per-profile format kept writing simple and made each profile independently checkable. Making every slow task (LCD, LED, button) a small piece of work inside the common `service()` loop kept the motor timing exact without needing interrupts. The main limitation is that the controller is open-loop: the LCD shows the commanded speed, not the measured speed, which is why the motor model's inertia had to be set realistically.

## 10. APPLICATIONS AND FUTURE IMPROVEMENTS

### 10.1 Applications

- **Washing machines:** wash (agitation) and spin-dry cycles, as in the WASH and SPIN profiles.
- **Gates, garage doors and window blinds:** open–wait–close sequences with soft start and stop.
- **Fans and exhaust fans:** natural-breeze and timed speed patterns.
- **Kitchen appliances:** blender and mixer pulse modes.
- **Conveyors, turntables and toys:** any repeated motion pattern that may need to change later.

### 10.2 Limitations and Future Improvements

- Add a speed sensor (encoder or Hall sensor) and a PI controller for closed-loop speed control.

- Let the user edit and store profiles from a keypad or over UART from a PC.
- Use hardware PWM at 20 kHz on real hardware to make the motor quieter.
- Add current sensing for stall and overload protection, and a watchdog timer.
- Protect each page with a CRC-16 and keep two copies so that a power cut during a write cannot lose a profile.
- Build and test the circuit on a real LPC2148 board.

## 11. CONCLUSION

An SPI EEPROM motor-profile controller based on the LPC2138 (and compatible with the LPC2148) was designed, programmed in C with Keil µVision and verified in Proteus. Six practical motor profiles — demonstration, washing-machine wash and spin, gate, natural-breeze fan and blender pulse — are stored in a 25LC256 SPI EEPROM, one per 64-byte page with a version, id and checksum, and are restored automatically if a page is blank or corrupt. The motor is driven through an L293D with 100 Hz PWM, linear ramps and a 200 ms dead time before every reversal. A 16×2 LCD shows the profile, direction, speed, phase, step and repeat, and the LED gives a different pattern for forward, reverse, stop and ramping. The EEPROM contents in Proteus matched the firmware's records byte for byte, the motor followed every profile, and emulation confirmed that the full 88.2 s sequence runs with exact timing and no drift. The next steps are closed-loop speed control and testing on real hardware.

## REFERENCES

[1] NXP Semiconductors, *UM10120: LPC2131/2/4/6/8 User Manual*, Rev. 4, 23 April 2012.

[2] NXP Semiconductors, *UM10139: LPC214x User Manual* (LPC2148).

[3] Microchip Technology Inc., *25AA256/25LC256 256K SPI Bus Serial EEPROM*, data sheet DS20001822J.

[4] Texas Instruments, *L293x Quadruple Half-H Drivers*, data sheet.

[5] Hitachi, *HD44780U (LCD-II) Dot Matrix Liquid Crystal Display Controller/Driver*, data sheet.

[6] Arm Ltd. / Keil, *µVision IDE and ARM Compiler* user guides.

[7] Labcenter Electronics, *Proteus Design Suite* documentation.